



Curso IMPY Tech

Aula 02

Controle de fluxo e códigos funcionais

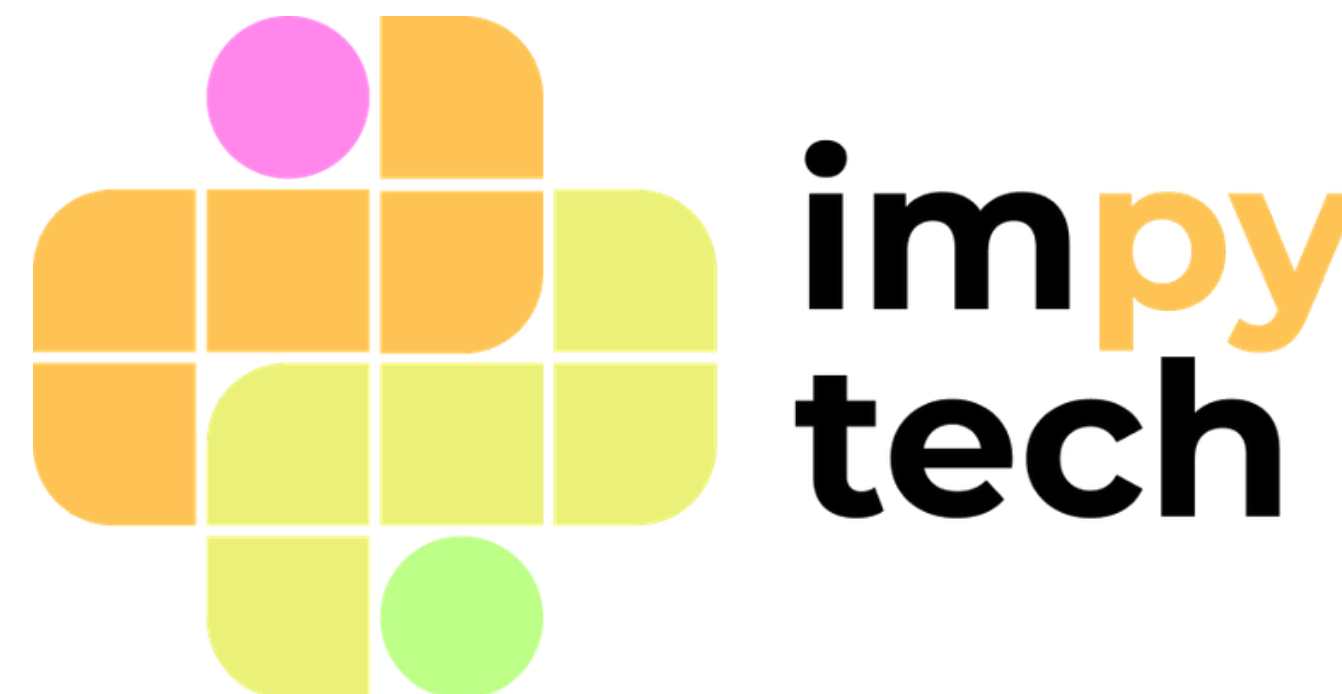
Ryan Kevin

Bruno Pereira

Gustavo Cardoso

Pedro Alberti

Cristiane Sampaio



01

Lógica proposicional

- Proposições
- Conectivos lógicos

02

Variáveis Booleanas

- Operadores lógicos
- Comparadores lógicos
- Condicionais (if, elif, else)

03

Laços de repetição

- *for*
- *while*

05

Funções

- Criação de funções
- Chamada de funções
- Parâmetros
- Tipos de retorno

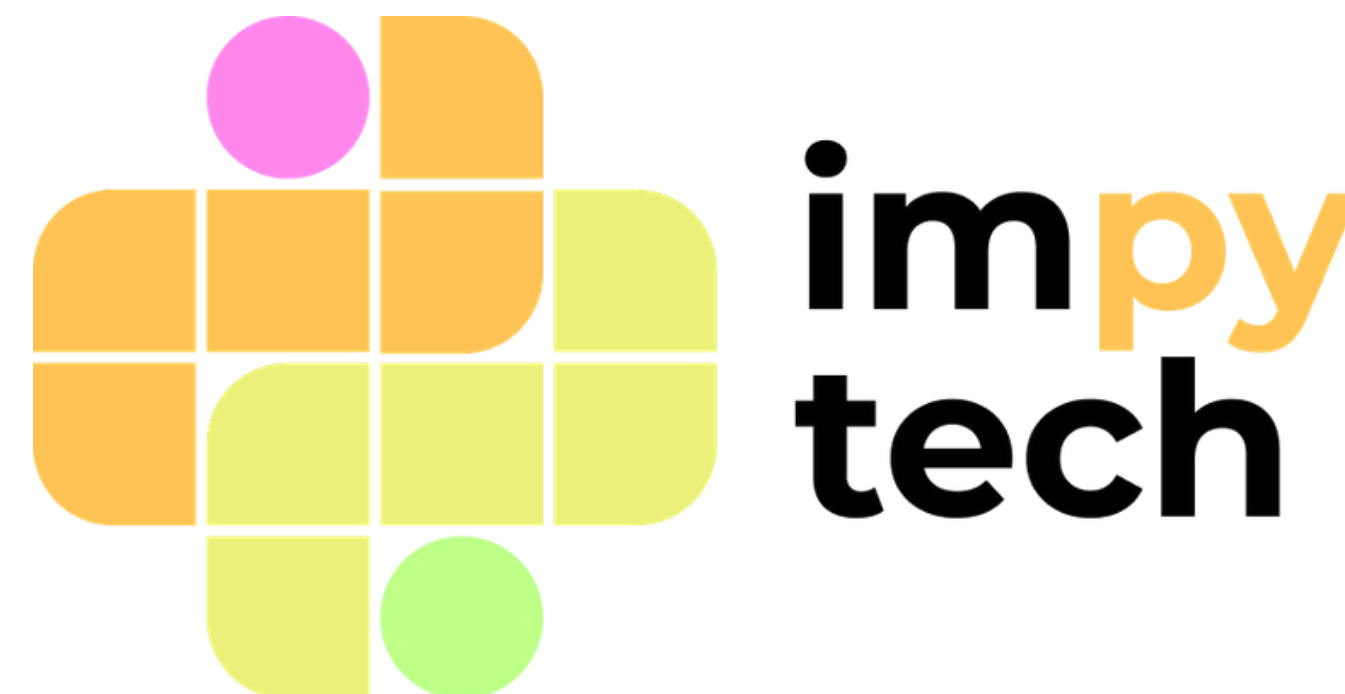
04

Bibliotecas

- Uso do “import” e “from”
- Bibliotecas internas
 - *math, time, random*
- Bibliotecas externas
 - Uso do “pip”
 - Documentação



Lógica Proposicional



Sentenças lógicas (Proposições)

“...é uma declaração com sentido completo que pode ser classificada de forma inequívoca como **verdadeira (V)** ou **falsa (F)**”

Exemplos

“O céu é azul” → **Proposição verdadeira (V)**

“A terra é plana” → **Proposição Falsa (F)**

“Que dia lindo!” → **Não é proposição:** Uma exclamação, não pode ser classificada como V ou F

“Que horas são?” → **Não é proposição:** Uma pergunta, não pode ser classificada como V ou F

“A cor azul é mais bonita” → **Não é proposição:** Algo subjetivo, uma opinião.

Sentenças lógicas (Proposições)

Podemos usar letras para representar proposições

Exemplos

“Pedro está com dor de cabeça = **P**

“Pedro está com febre” = **Q**

Proposições abertas

Podemos também ter proposições que dependem de alguma **variável**, onde o valor lógico da proposição (**V** ou **F**) depende dessa variável

Exemplos

Seja a variável **T** = “Temperatura de Pedro”, podemos construir a sentença aberta:

P = “A temperatura de Pedro é maior que 39 graus”

- Se **T** > **39°**, então P é **verdadeira**
- Se **T** ≤ **39°**, então P é **falsa**

Conectivos lógicos

Para conectar duas proposições em uma única, podemos utilizar os **conectivos lógicos**:

Exemplos

“Pedro está com dor de cabeça = **P**”

“Pedro está com febre” = **Q**

“Pedro está com dor de cabeça **OU** Pedro está com febre” $\rightarrow P \text{ **OU** } Q$

“Pedro está com dor de cabeça **E** Pedro está com febre” $\rightarrow P \text{ **E** } Q$

“Pedro **NÃO** está com dor de cabeça” $\rightarrow \text{**NÃO** } P$

Conectivo “OU”

“Pedro está com dor de cabeça **OU** Pedro está com febre”

Estou afirmando que Pedro está com **pelo menos um dos dois** sintomas:

- Se Pedro está com dor de cabeça, mas não está com febre, a sentença é **verdadeira**
- Se Pedro não está com dor de cabeça, mas está com febre, a sentença é **verdadeira**
- Se Pedro está com dor de cabeça **E** está com febre, a sentença também é **verdadeira**
- Se Pedro **não está** com dor de cabeça, e **não está** com febre, a sentença é **falsa**

Conectivo “E”

“Pedro está com dor de cabeça **E** Pedro está com febre”

Estou afirmando que Pedro está com **os dois sintomas simultaneamente**:

- Se Pedro está com dor de cabeça e com febre, a sentença é **verdadeira**
- Se Pedro está com um sintoma, mas não estiver com outro, a sentença é **falsa**
- Se Pedro não está com nenhum sintoma, então a sentença é **falsa**

Conectivo “NÃO”

“Pedro **NÃO** está com dor de cabeça”

Estou apenas “**trocando**” o valor da sentença original **P**:

- Se Pedro não está com dor de cabeça (**P falso**), então essa sentença é **verdadeira (não P verdade)**
- Se Pedro está com dor de cabeça (**P verdade**), então essa sentença é **falsa (não P falso)**

Variáveis booleanas

“Proposições” em Python

Para trabalhar a lógica proposicional em Python, utilizamos as **variáveis booleanas**.

Variáveis do tipo “**bool**” traduzem os valores **V** e **F** das proposições para **True (1)** e **False (0)** (sempre capitalizado!!!)

Tente definir as variáveis abaixo e use o print()

```
P = True # Tente também usar bool(1)
Q = False # Tente também usar bool(0)
print(P)
print(Q)
```

Conectivo	Python	Funcionamento	Exemplo
“OU”	or	Retorna True se ao menos um dos dois valores for True	P or Q
“E”	and	Retorna True apenas se os dois valores forem True	P and Q
“NÃO”	not	Sempre retorna o valor oposto	not P

Exemplo

```
P = True
Q = False
print(P and Q) # True and False
print(P or Q)  # True or False
print(not P)   # not True
print(not Q)   # not False
```

Output

```
False
True
False
True
```

Sentenças abertas

Geralmente, queremos criar um **valor booleano** (verdadeiro ou falso) com base em alguma “pergunta” envolvendo outras variáveis (geralmente números). Isto é, queremos usar uma **sentença aberta**.

No exemplo anterior, queremos saber se a temperatura de Pedro é superior a 37°. Podemos escrever isso como:

Exemplo

```
T = float(input("Qual a temperatura de Pedro?"))  
P = T > 37  
print(P)
```

Comparadores lógicos

Comparador	Python	Exemplo 1	Resultado 1	Exemplo 2	Resultado 2
Igualdade	<code>==</code>	<code>1 == 2</code>	False	<code>1 == 1</code>	True
Diferente	<code>!=</code>	<code>1 != 2</code>	True	<code>1 != 1</code>	False
Maior	<code>></code>	<code>2 > 1</code>	True	<code>2 > 2</code>	False
Maior ou igual	<code>>=</code>	<code>2 >= 1</code>	True	<code>2 >= 2</code>	True
Menor	<code><</code>	<code>1 < 3</code>	True	<code>1 < 1</code>	False
Menor ou igual	<code><=</code>	<code>1 <= 3</code>	True	<code>1 <= 1</code>	True

Condicionais

Podemos indicar para que o nosso algoritmo realize algo apenas se uma certa **condição** for satisfeita. Para isso, devemos criar uma condição usando **variáveis booleanas**, **conectivos** e **comparadores** (não necessariamente todos ao mesmo tempo)

Por exemplo: com base na temperatura **T** de Pedro, você deseja mostrar essa informação:

- Se a **T** estiver acima de **37°** graus, você classificá-o como **“acima do normal”**
- Caso contrário, você classificá-o como **“normal”**

Para construir um algoritmo desse tipo, utilizamos as **condicionais**

Conditional if else

```
T = float(input("Qual a temperatura de Pedro?"))

if (T > 37):
    situacao = "Acima do normal"

else:
    situacao = "Normal"

print(situacao)
```

Criamos um bloco no estilo:

Condicional if else

```
if( proposição ):
    (sub-código A)

else:
    (sub-código B)
```

- Se a proposição (ou condição) passada para o if for **verdadeira**, então o código irá executar o **sub-código A**.
- Caso a proposição seja **falsa**, o código irá executar o **sub-código B**.

E quando temos mais condições?

Com base na temperatura **T** de Pedro, você deseja mostrar essa informação:

- Se a temperatura dele estiver acima de **37°** graus, você classificá-o como **“acima do normal”**
- Caso a temperatura dele estiver **abaixo** de **35°**, você classifica-o como **“abaixo do normal”**
- Caso contrário, você classificá-o como **“normal”**

Condicionais múltiplas: elif

```
T = float(input("Qual a temperatura de Pedro?"))

if (T > 37):
    situacao = "Acima do normal"

elif(T < 35):
    situacao = "Abaixo do normal"

else:
    situacao = "Normal"

print(situacao)
```

- **Alguns pontos importantes sobre as condicionais:**
 - O bloco **if()** SEMPRE deve ser o primeiro
 - O bloco **if()** SEMPRE deve ter uma condição
 - Não há limites para a quantidade de **elif()** para um dado **if()**
 - O bloco **else()** não é sempre obrigatório
 - O bloco **else()** SEMPRE deve estar após **if()** e todos os **elif()**

E se, ao invés de precisarmos classificar **apenas a situação do Pedro**, precisássemos classificar a situação de **5 pacientes diferentes**

Precisaríamos rodar o nosso código 5 vezes?

E se, ao invés de precisarmos classificar **apenas a situação do Pedro**, precisássemos classificar a situação de **5 pacientes diferentes**

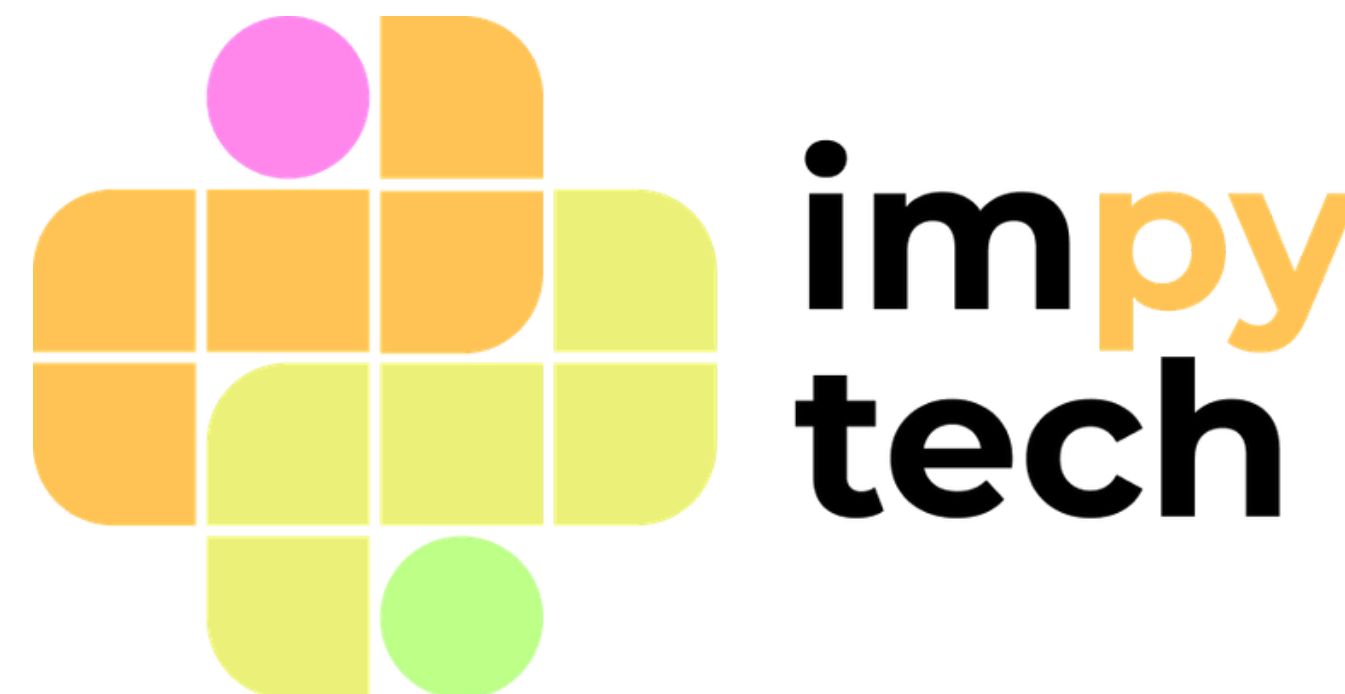
Precisaríamos rodar o nosso código 5 vezes?

A resposta é: **NÃO**

Temos métodos para rodar códigos múltiplas vezes



Laços de repetição



Laços “for”

Para executar um pedaço de código **um número fixo de vezes** (ex. 5), utilizando os laços **for**.

No exemplo abaixo, podemos determinar o número de vezes que queremos que um certo trecho de código se repita

Exemplo

```
n = 3 # Tente mudar o número de repetições

for c in range(n): # Será executado n vezes
    print("Olá!")
```

Exemplo

```
...  
for c in range(n):  
....
```

O que é esse “c”?

Variável de controle

No exemplo acima, a variável “c” é chamada de **variável de controle**

“...é um contador ou sinalizador usado para monitorar, contar ou ditar o momento em que o laço deve ser encerrado”

Laços de repetição - for

Variável de controle

```
n = 5

for c in range(n):
    print(c)
```

Output

```
0
1
2
3
4
```

Laços de repetição - for

Variável de controle

```
n = 5

for c in range(n):
    print(c ** 2)
```

Output

```
0
1
4
9
16
```

Exemplo

```
...  
for c in range(n):  
....
```

E esse “**range()**”?

Controlador de passo

A estrutura **range()** define como os índices da **variável de controle** vai se comportar. Por exemplo:

- “Quero que meu **c** comece em 10 e termine em 15”
- “Quero que meu **c** vá até 10 pulando de 2 em 2”

Controlador de passo

Podemos passar até três parâmetros para o **range()**, que são:

- Início: em que número **c** irá iniciar?
- fim: em que número **c** irá parar?
- passo: de quantos em quantos números meu **c** deve “pular”?

Controlador de índice

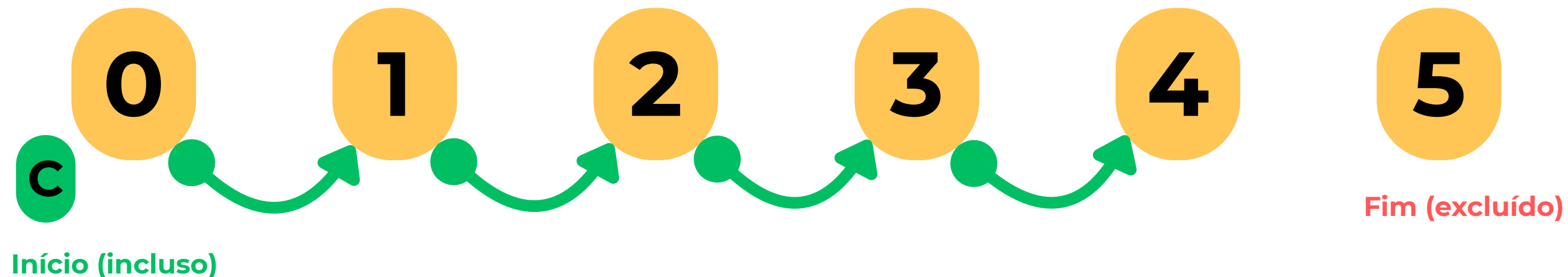
```
...  
for c in range(inicio, fim, passo):  
....
```

IMPORTANTE!

- No controlador **range()**, o índice de início é **INCLUSIVO**
- No controlador **range()**, o índice de fim é **EXCLUSIVO**

Controlador de índice

```
for c in range(0, 5, 1):  
    print(c)
```



Controlador de índice

```
for c in range(1, 5, 1):  
    print(c)
```

Fora do intervalo

0

1

2

3

4

5

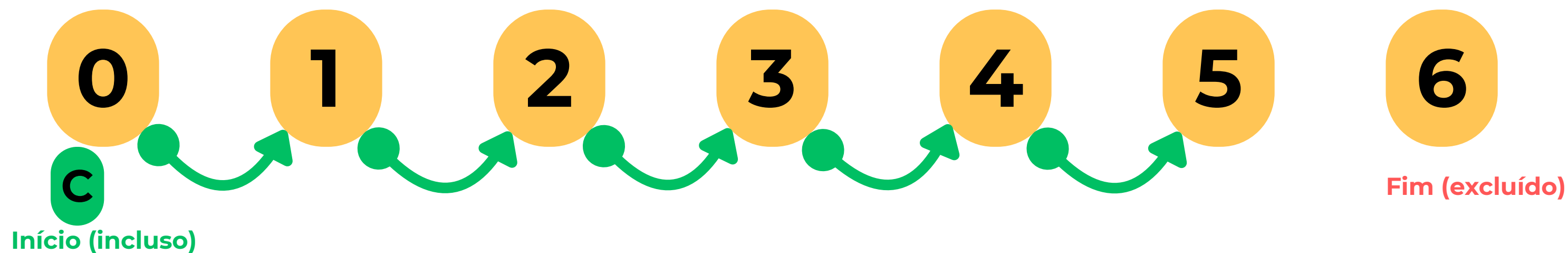
Fim (excluído)

c

Início (incluso)

Controlador de índice

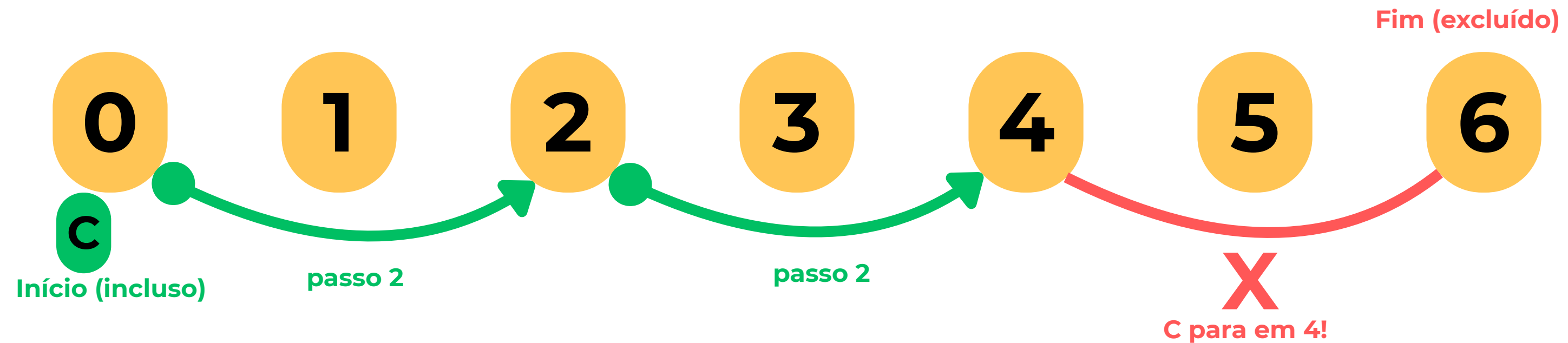
```
for c in range(0, 6, 1):  
    print(c)
```



Laços de repetição - for

Controlador de índice

```
for c in range(0, 6, 2):  
    print(c)
```



Experimente você mesmo

```
inicio = 0 # Troque por outro valor
fim = 7    # Troque por outro valor
passo = 2  # Troque por outro valor

for c in range(inicio, fim, passo):
    print(c)
```

Classificador de paciente

Temos um código para classificar um paciente de acordo com sua temperatura. **E se quisermos classificar 5 pacientes?**

Classificador (paciente único)

```
T = float(input("Qual a temperatura de Pedro?"))

if (T > 37):
    situacao = "Acima do normal"
elif(T < 35):
    situacao = "Abaixo do normal"
else:
    situacao = "Normal"

print(situacao)
```

Classificador (múltiplos pacientes)

```
for c in range(5):    # Repete o programa cinco vezes

    T = float(input("Qual a temperatura do Paciente?"))

    if (T > 37):
        situacao = "Acima do normal"
    elif(T < 35):
        situacao = "Abaixo do normal"
    else:
        situacao = "Normal"

    print(situacao)
```

- **Alguns pontos importantes sobre o laço for():**
 - Os parâmetros **início** e **passo** são **opcionais** (como visto no slide anterior). Quando eles não são fornecidos, o Python considera **início=0** e **passo=1**
 - Podemos passar **início** e **fim**, sem informar **passo**. (Padrão 1)
 - Podemos dar **qualquer nome** para a variável de controle

Pontos importantes

```
for paciente in range(5):          # início=0 (padrão) / fim=5 / passo=1 (padrão)
    print(paciente)

for temperatura in range(0,5):     # início=0 / fim=5 / passo=1 (padrão)
    print(temperatura)
```

E se, ao invés de precisarmos classificar **apenas 5 pacientes**, precisássemos classificar a situação de **um número desconhecido de pacientes**?

Precisaríamos usar um laço **for()** e torcer pra acertarmos o número de pacientes?

E se, ao invés de precisarmos classificar **apenas 5 pacientes**, precisássemos classificar a situação de **um número desconhecido de pacientes**?

Precisaríamos usar um laço **for()** e torcer pra acertarmos o número de pacientes?

A resposta é: **NÃO**

Também temos métodos para casos assim

Laços “while”

Os laços **while** executam um trecho de código **indefinidamente enquanto uma dada condição for verdadeira**

Laços de repetição - while

Laços while

```
n = 0

while(n<3):
    n = n + 1
    print(n)

print("Fim")
```

Output

```
1
2
3
Fim
```

Laços de repetição - while

Laços while

```
n = 0

while(n<3):    # Condição: n < 3
    n = n + 1
    print(n)

print("Fim")
```

1º Passo:

$n = 0$

$0 < 3 ? \rightarrow \text{True}$

$n = 0 + 1$

$n = 1$

2º Passo:

$n = 1$

$1 < 3 ? \rightarrow \text{True}$

$n = 1 + 1$

$n = 2$

3º Passo:

$n = 2$

$2 < 3 ? \rightarrow \text{True}$

$n = 2 + 1$

$n = 3$

4º Passo:

$n = 3$

$3 < 3 ? \rightarrow \text{False}$

`print("Fim")`

Laços “while”

E se, ao invés de precisarmos classificar apenas 5 pacientes, **precisássemos classificar a situação de um número desconhecido de pacientes?**

Defina uma temperatura **T = 0** arbitrária que indicará o **fim do seu programa**, e crie uma **condição** para quando seu **T** for diferente disso:

Classificador (usando while)

```
T = float(input("Qual a temperatura do Paciente?"))

while T != 0: # Digite 0 para sair do laço while

    T = float(input("Qual a temperatura do Paciente?"))

    if (T > 37):
        situacao = "Acima do normal"
    elif(T < 35):
        situacao = "Abaixo do normal"
    else:
        situacao = "Normal"

    print(situacao)

print("Fim do programa")
```

Break

Para sair do laço **while** mesmo com a sua condição sendo verdadeira, podemos usar o comando **break** dentro do bloco de código.

O comando **break** forçará o código a sair do laço, independente da situação

Laços de repetição - while

Laços while

```
n = 0

while(n < 100):
    n = n + 1
    print(n)

    if(n == 3):
        break

print("Fim")
```

1º Passo:

`n = 0`

0 < 100 ? → True

0 == 3? → False

`n = 1`

2º Passo:

`n = 1`

1 < 100 ? → True

1 == 3? → False

`n = 2`

1º Passo:

`n = 2`

2 < 100 ? → True

2 == 3? → False

`n = 3`

1º Passo:

`n = 3`

3 < 100 ? → True

3 == 3? → True

`break`
`print("Fim")`

CUIDADO!

- Usar o laço **while()** requer uma camada extra de **cuidado**. Utilizar condições em um código que essa condição **sempre será verdadeira**, fará o seu código ficar rodando **infinitamente**, causando travamento.

Exemplo de “código infinito”

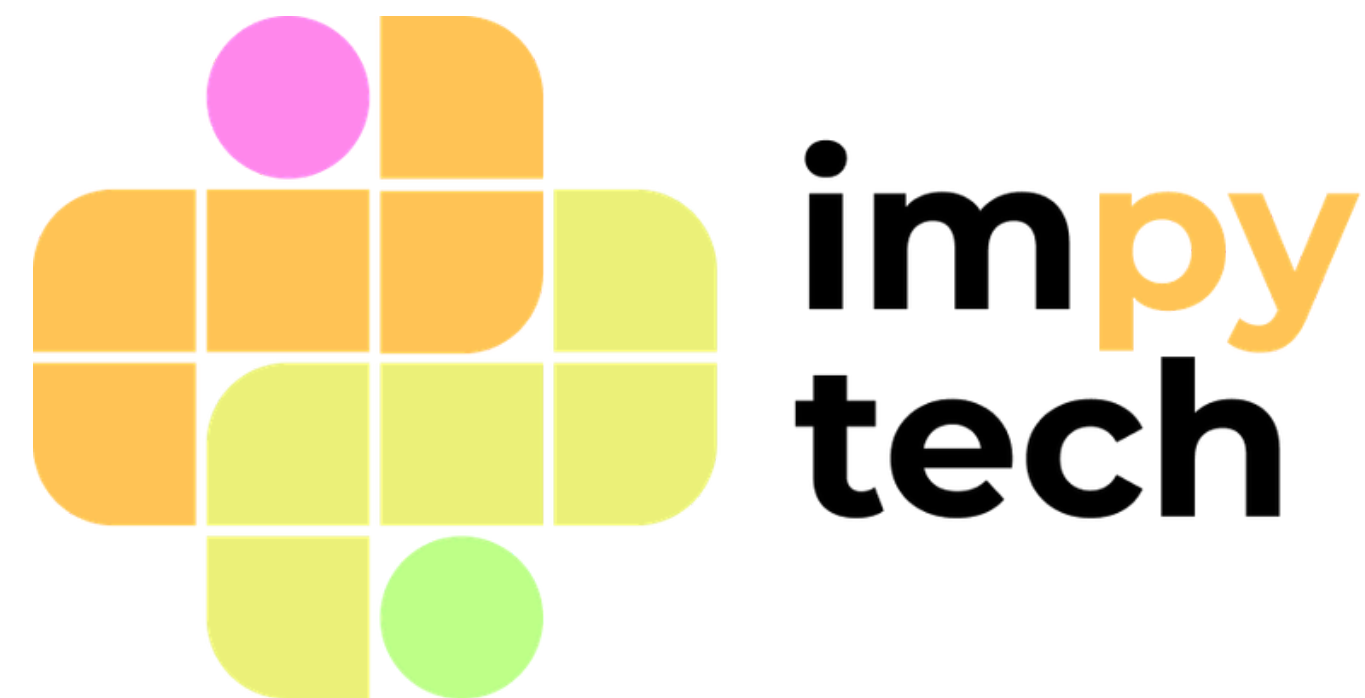
```
n = 0

while(n<3):
    print(n) # n não está sendo atualizado

print("Fim")
```



Funções



Criando funções

E se, quisermos que uma determinada lógica seja executada várias vezes ao longo do código? Precisaríamos escrever essa mesma lógica várias vezes? **Não**

Para situações assim, podemos criar uma **função**. No exemplo abaixo, criamos uma função “**cumprimentar()**” que exibe “Olá mundo!” na tela sempre que é usada.

Funções

```
def cumprimentar():  
    print("Olá mundo!")
```

Funções

```
def cumprimentar():  
    print("Olá mundo!")
```

```
cumprimentar()  
cumprimentar()  
cumprimentar()
```

Output

```
Olá mundo!  
Olá mundo!  
Olá mundo!
```

Funções

```
def cumprimentar():  
    print("Olá mundo!")  
def tracos():  
    print("-----")  
  
tracos()  
cumprimentar()  
tracos()
```

Output

```
-----  
Olá mundo!  
-----
```

Parâmetros

Na maioria das vezes, precisaremos utilizar as **funções** baseadas em um (ou mais) **parâmetro(s)** fornecidos. Para isso, podemos configurar os parâmetros que serão **passados** para a função quando for chamada

Um parâmetro funciona como uma **variável** que será usada **dentro da função**, e depois “excluída”

Funções

```
def somar(n1, n2) # Dois parâmetros: n1 e n2
    print(n1 + n2)
```

Funções

```
def somar(n1, n2) # Dois parâmetros: n1 e n2
    print(n1 + n2)
```

```
somar(1,1)
somar(10,9)
somar(-5,5)
```

Output

```
2
19
0
```

Erro

```
def somar(n1, n2): # Dois parâmetros: n1 e n2
    print(n1 + n2)

soma = somar(1,1) # O valor (2) irá aparecer pois a função de soma EXIBE a soma dos números
print(soma)       # Essa linha irá exibir "None"
print(soma + 1)   # Essa linha gerará um erro
```

Output

```
2
None
TypeError ...
```

Retorno de valores

Para que possamos “olhar para uma função como um valor”, precisamos sempre que essa função **retorne** algo.

Nos exemplos anteriores, a nossa função estava apenas **exibindo** algo na tela. Se quisermos que uma variável tenha o valor da **soma de dois números**, precisamos que essa função **retorne a soma desses números**.

return

```
def somar(n1, n2) # Dois parâmetros: n1 e n2
    return (n1 + n2)
```

Correção com return

```
def somar(n1, n2) # Dois parâmetros: n1 e n2
    return (n1 + n2) # Retorna a soma de dois números, ao invés de só exibir

soma = somar(1,1)
print(soma) # Diferente do caso anterior, o valor (2) irá aparecer por causa dessa linha
print(soma + 1)
```

Output

2
3

Bibliotecas

Bibliotecas funcionam como “conjuntos de funções”, que podemos baixar e utilizar no nosso código diretamente, sem a necessidade de inseri-las no código.

Biblioteca: Matemática

somar()

fatorial()

subtrair()

raiz_quadrada()

multiplicar()

dividir()

integrar()

...

Biblioteca: Análise de dados

media()

ordenar()

moda()

calcular_p_valor()

mediana()

ler_csv()

simular()

...

Biblioteca: Gerar gráficos

histograma()

grafico()

grafico_barra()

aproximacao()

...

Importando bibliotecas

Por enquanto, utilizaremos apenas **bibliotecas nativas do Python** (já vem instaladas, e podem ser usadas facilmente)

Temos duas opções de **importação**:

- Importar a biblioteca inteira usando **import**
- Importar alguma(s) função(ões) específica(s) usando **from ... import**

Importando bibliotecas

```
import math                # Importa toda a biblioteca math (todas as funções)
from random import randint # Importa a função randint() da biblioteca random
import numpy as np         # A biblioteca numpy agora pode ser chamada usando apenas np
```

Documentação

Para entendermos as funções de uma dada biblioteca, precisamos ler suas **documentações**, que geralmente podem ser encontradas apenas pesquisando na internet.

A documentação conta com a explicação completa de como usar cada função da biblioteca. Por enquanto, tente utilizar apenas algumas funções básicas:

Importando bibliotecas

```
import math
from random import randint

print(math.pi)          # Acessa o valor "pi" da biblioteca "math", que retorna uma aproximação de pi
print(randint(1,10))    # Usa a função "randint()" para gerar um inteiro aleatório entre 1 e 10
```

Referências bibliográficas:

- Pense em Python **(1)**: Capítulo **3 ; 5 ; 6 e 7**
- Python for Data Analysis (2): Seção **3.2**

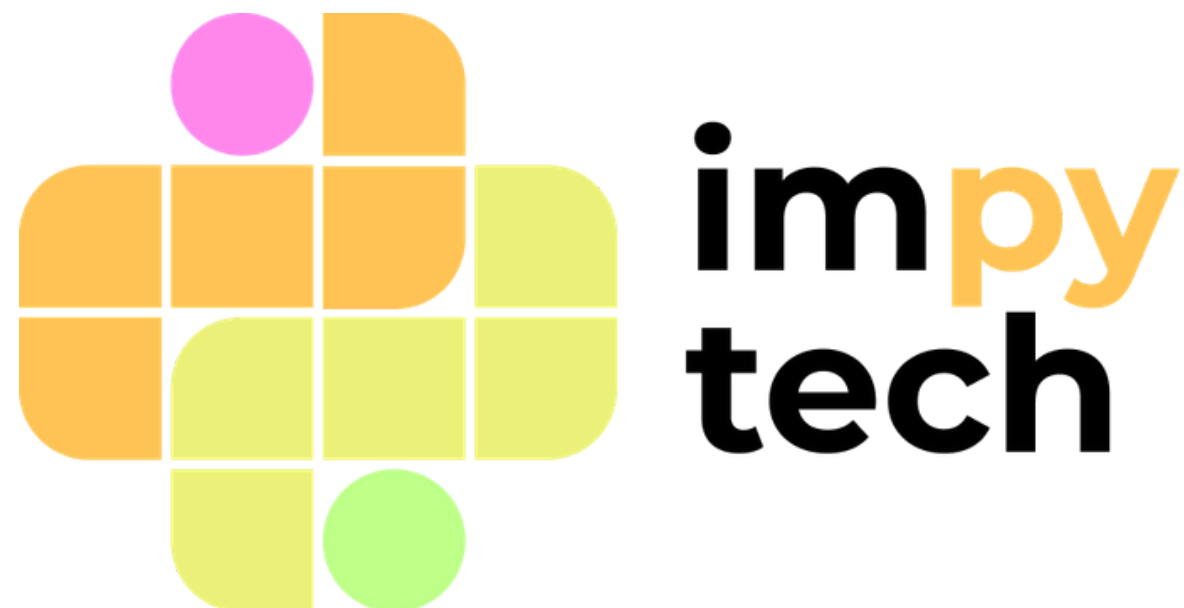
Leituras Adicionais

- aula02_EXTRA_aprofundamento.ipynb (**GitHub**)

Exercícios

- aula02_exercicios.ipynb





Obrigado!

Avalie a aula



Ryan Kevin

al.ryan.felinto@impatech.edu.br

al.bianca.abreu@impatech.edu.br

al.bruno.paula@impatech.edu.br

al.gustavo.cardoso@impatech.edu.br

al.pedro.alberti@impatech.edu.br

al.cristiane.sampaio@impatech.edu.br